ET TU, BRUTE?

BREAKING A CAESAR CIPHER IN BASH · PORTFOLIO ACTIVITY · LINUX FUNDAMENTALS

ANALYST	ROLE	DATE	REFERENCE
Kelly Guzman	**Founder & SOC Analyst, CTA**	**May 13, 2026**	**Bash · Crypto Lab**

Scenario

A crypto exercise presented two encrypted artifacts on a Bash command line. The first was a Caesar-shifted text file containing hidden recovery instructions. The second was an AES-256-CBC encrypted file (`Q1.encrypted`) that could not be opened without the passphrase concealed inside the first file. The task: break the Caesar cipher, extract the AES passphrase from the recovered plaintext, then decrypt the protected file using `openssl`.

Methodology

- **Identify the cipher.** The filename `.leftShift3` signaled a Caesar cipher with a left shift of three. Each letter in the ciphertext had been shifted three positions backward in the alphabet, so the inverse transformation was a right shift of three.
- **Reverse the shift with tr.** Piped the ciphertext through `tr` with character-class mapping to apply the inverse Caesar shift in one command, producing readable plaintext containing the AES decryption instructions and passphrase.
- **Decrypt the AES payload.** Used `openssl aes-256-cbc` with PBKDF2 key derivation, base64 input flag, and the recovered passphrase to decrypt `Q1.encrypted` into `Q1.recovered`.
- **Verify the result.** Read the recovered file with `cat` to confirm successful decryption and validate the plaintext output.

Commands used

Caesar cipher inverse shift, reading the hidden file and mapping it back to plaintext:

```
cat .leftShift3 | tr "a-zA-Z" "d-za-cD-ZA-C"
```

AES-256-CBC decryption using the passphrase revealed by the Caesar step:

```
openssl aes-256-cbc -pbkdf2 -a -d -in Q1.encrypted -out Q1.recovered -k ettubrute
```

Verification:

```
cat Q1.recovered
```

Terminal capture

```
analyst@b542be81dbaf:~/caesar$
" "a-zA-Z"42be81dbaf:~/caesar$ cat .leftShift3 | tr "d-za-cD-ZA-C"
In order to recover your files you will need to enter the following command:

openssl aes-256-cbc -pbkdf2 -a -d -in Q1.encrypted -out Q1.recovered -k ettubrute
analyst@b542be81dbaf:~/caesar$
" "a-zA-Z"42be81dbaf:~/caesar$ cat .leftShift3 | tr "d-za-cD-ZA-C"
In order to recover your files you will need to enter the following command:

openssl aes-256-cbc -pbkdf2 -a -d -in Q1.encrypted -out Q1.recovered -k ettubrute
analyst@b542be81dbaf:~/caesar$
analyst@b542be81dbaf:~/caesar$ cd ~
encrypted -out Q1.recovered -k ettubrutecbc -pbkdf2 -a -d -in Q1.e
analyst@b542be81dbaf:~$ ls
Q1.encrypted  Q1.recovered  README.txt  caesar
analyst@b542be81dbaf:~$ cat Q1.recovered
If you are able to read this, then you have successfully decrypted the classic cipher text. Yo
u recovered the encryption key that was used to encrypt this file. Great work!
analyst@b542be81dbaf:~$
```

Result

Both decryption stages succeeded. The Caesar-shifted file revealed the AES command and passphrase. The AES decryption produced the original plaintext, which confirmed recovery of the protected file and validated the full attack chain end-to-end.

Reflection

This exercise chained two cryptographic techniques from opposite ends of the historical timeline. The Caesar cipher is trivially breakable today but illustrates the substitution-cipher logic that underpins more sophisticated systems. AES-256-CBC with PBKDF2 represents current standard symmetric encryption, used in file protection, VPN tunnels, and full-disk encryption.

Working both stages back-to-back reinforced a security principle that maps directly onto incident response: an attacker who breaks one layer often gains the credentials needed to break the next. The same pattern shows up in real engagements as credential stuffing,

password reuse across systems, and post-compromise credential harvesting that opens lateral movement paths. Recognizing the chain matters as much as breaking any single link.

The exercise also reinforced practical Bash fluency. The `tr` character-class transform is a one-line solution to a problem that in another language would take a loop and explicit alphabet handling. That kind of leverage is exactly why Linux command-line proficiency remains foundational for analyst work.